

PROYECTO PRÁCTICAS AULA 24

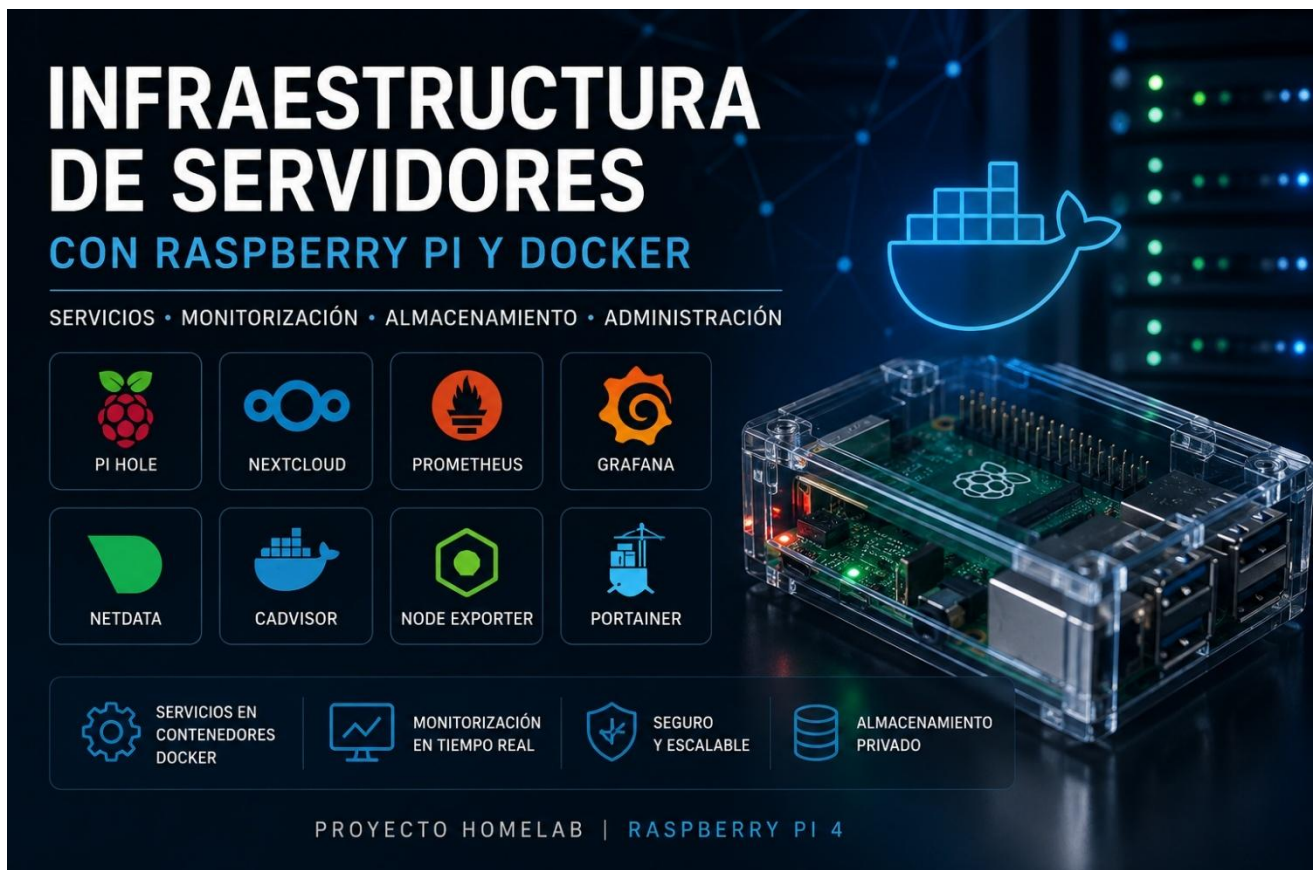
Grupo 5: Ivan, Santiago, Sergio R.

Grupo 6: Yuneida, Cristina.

Grupo 7: Sergio V, Felipe y Fernando.

Ciclo Formativo de Grado Superior en Administración de Sistemas
Informáticos en Red (ASIR) – Año 2025/2026.

Infraestructura de Servicios y Monitorización con Raspberry Pi 4, Docker



**INFRAESTRUCTURA
DE SERVIDORES**
CON RASPBERRY PI Y DOCKER

SERVICIOS • MONITORIZACIÓN • ALMACENAMIENTO • ADMINISTRACIÓN

 PI HOLE	 NEXTCLOUD	 PROMETHEUS	 GRAFANA
 NETDATA	 CADVISOR	 NODE EXPORTER	 PORTAINER

 SERVICIOS EN CONTENEDORES DOCKER	 MONITORIZACIÓN EN TIEMPO REAL	 SEGURO Y ESCALABLE	 ALMACENAMIENTO PRIVADO
---	---	--	--

PROYECTO HOMELAB | RASPBERRY PI 4

Índice:

Introducción	3
2. Objetivos del Proyecto	3
2.1 Objetivo General	3
2.2 Objetivos principales	3
2.3 Objetivos Técnicos	4
2.4 Objetivos de Aprendizaje	12
2.5 Tecnologías Utilizadas	12
3. Hardware Utilizado	13
Equipamiento principal	13
4. Arquitectura General	16
5. Instalación del Sistema Operativo	19
6. Instalación de Docker	22
7. Instalación de Portainer	25
8. Configuración de Pi hole	29
9. Creación de DNS Locales	32
Configuración de DNS locales	32
10. Configuración de Clientes Windows	34
Configuración manual del DNS	34
11. Monitorización	36
Grafana	37

Netdata	40
cAdvisor	41
Node Exporter	42
Prometheus	42
12. Seguridad Implementada	43
DNS local privado	44
13. Problemas Encontrados y Soluciones	47
14. Mejoras Futuras	51
15. Anexos	53
Conclusión	55

Introducción

Este proyecto consiste en el diseño e implementación de una infraestructura de red y servicios virtualizados utilizando una Raspberry Pi 4 como servidor principal.

La finalidad fue construir un entorno profesional de prácticas capaz de alojar múltiples servicios Docker, monitorización avanzada, DNS interno, almacenamiento web y herramientas de administración IT.

Toda la infraestructura fue desplegada y administrada utilizando Docker y Portainer, permitiendo centralizar la gestión de contenedores desde una interfaz web.

2. Objetivos del Proyecto

2.1 Objetivo General

El objetivo principal de este proyecto fue diseñar e implementar una infraestructura de servicios virtualizados utilizando una Raspberry Pi 4 como servidor principal dentro de una red local profesional.

La infraestructura fue desarrollada mediante contenedores Docker con el propósito de centralizar servicios, mejorar la administración del sistema y crear un entorno real de prácticas orientado a la administración de sistemas, redes y monitorización.

Además, se buscó construir una plataforma estable, escalable y de bajo consumo energético capaz de ejecutar múltiples servicios simultáneamente desde un único dispositivo.

2.2 Objetivos principales

- Instalar y configurar Raspberry Pi OS.
- Configurar Docker y Portainer.
- Desplegar múltiples servicios mediante contenedores.
- Configurar Pi-hole como servidor DNS interno.
- Implementar monitorización con Prometheus, Grafana, Netdata y cAdvisor.
- Crear almacenamiento web con Nextcloud.
- Implementar GLPI para gestión IT.
- Integrar toda la infraestructura dentro de una red local profesional.
- Gestionar servicios mediante contenedores Docker.

2.3 Objetivos Técnicos

Instalación y configuración del sistema operativo

Se realizó la instalación de Raspberry Pi OS como sistema operativo principal del servidor.

Posteriormente se configuraron los parámetros básicos del sistema:

Usuario administrador

Contraseña

Configuración de red

Acceso SSH

Actualización del sistema

Configuración del hostname

Esto permitió disponer de una base estable y segura sobre la que desplegar toda la infraestructura.

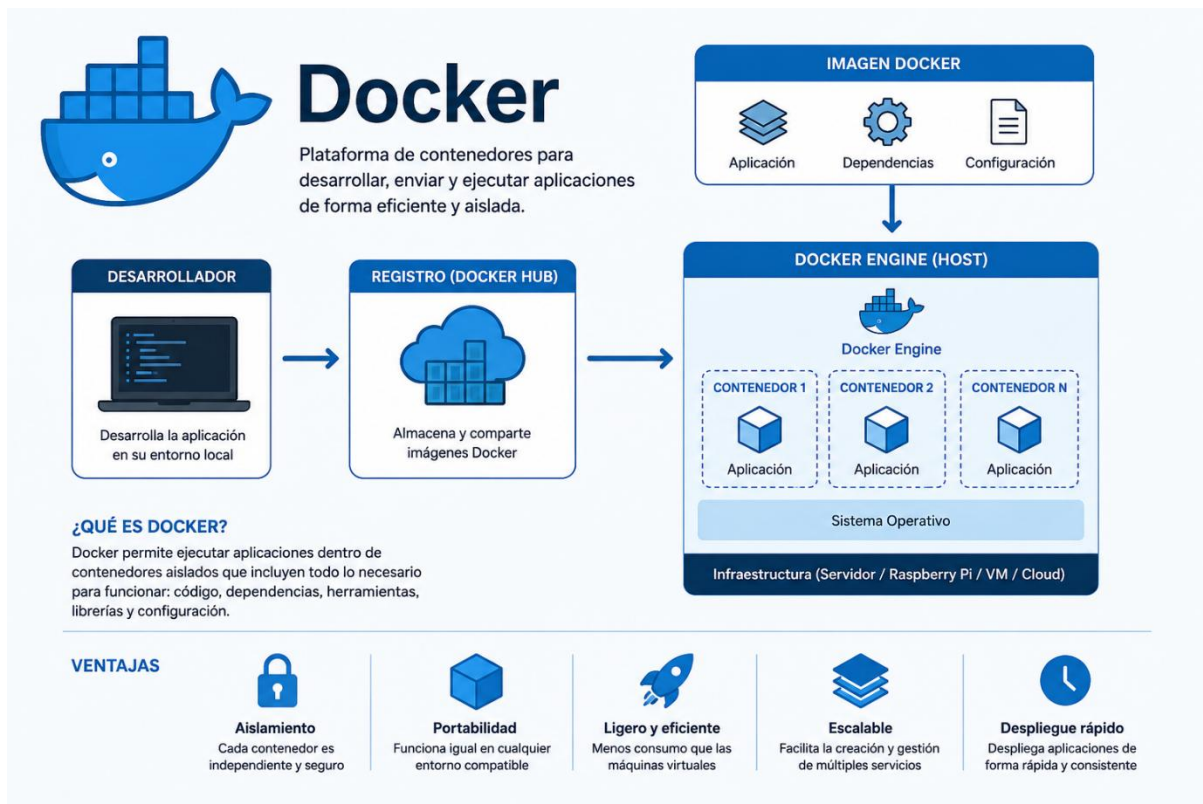
Implementación de Docker

Se instaló Docker Engine como plataforma de virtualización ligera para ejecutar todos los servicios mediante contenedores.

El uso de Docker permitió:

- Aislar servicios
- Reducir consumo de recursos
- Facilitar despliegues
- Mejorar la administración
- Centralizar aplicaciones

Cada servicio fue desplegado de manera independiente mediante imágenes Docker.



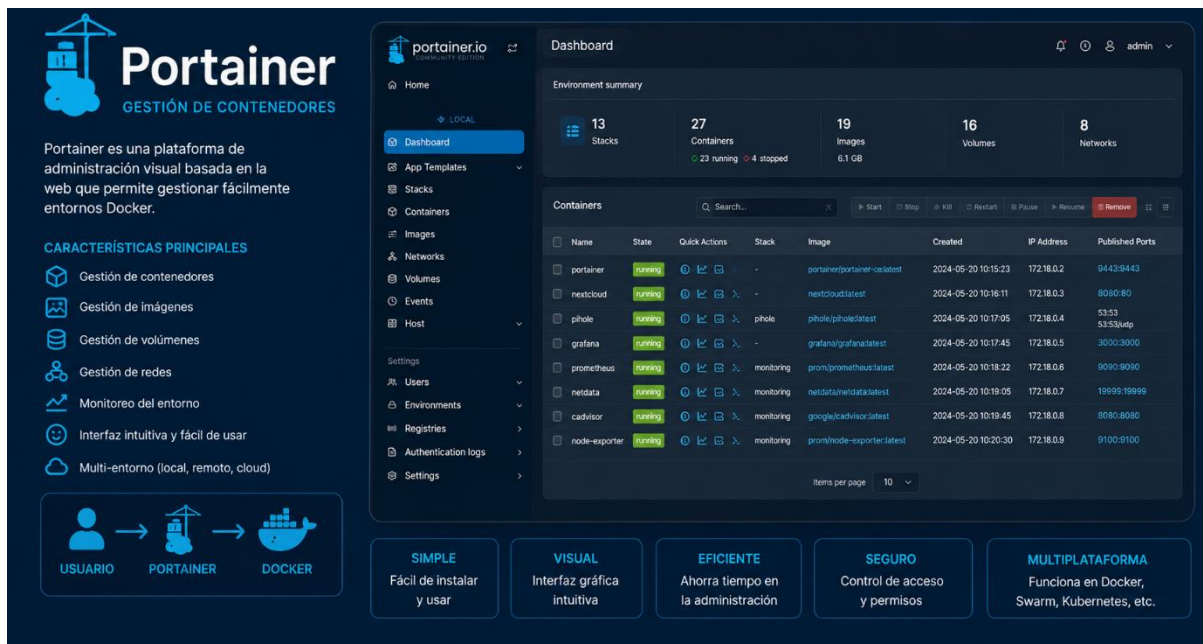
Gestión centralizada con Portainer

Se implementó Portainer como herramienta de administración visual de Docker.

Portainer permitió:

- Crear contenedores
- Gestionar imágenes
- Supervisar estados
- Revisar logs
- Reiniciar servicios
- Gestionar redes Docker
- Administrar volúmenes

Gracias a esta herramienta se simplificó completamente la gestión de la infraestructura desde interfaz web.



Implementación de Pi-hole como DNS interno

Se desplegó Pi-hole como servidor DNS interno de la red local.

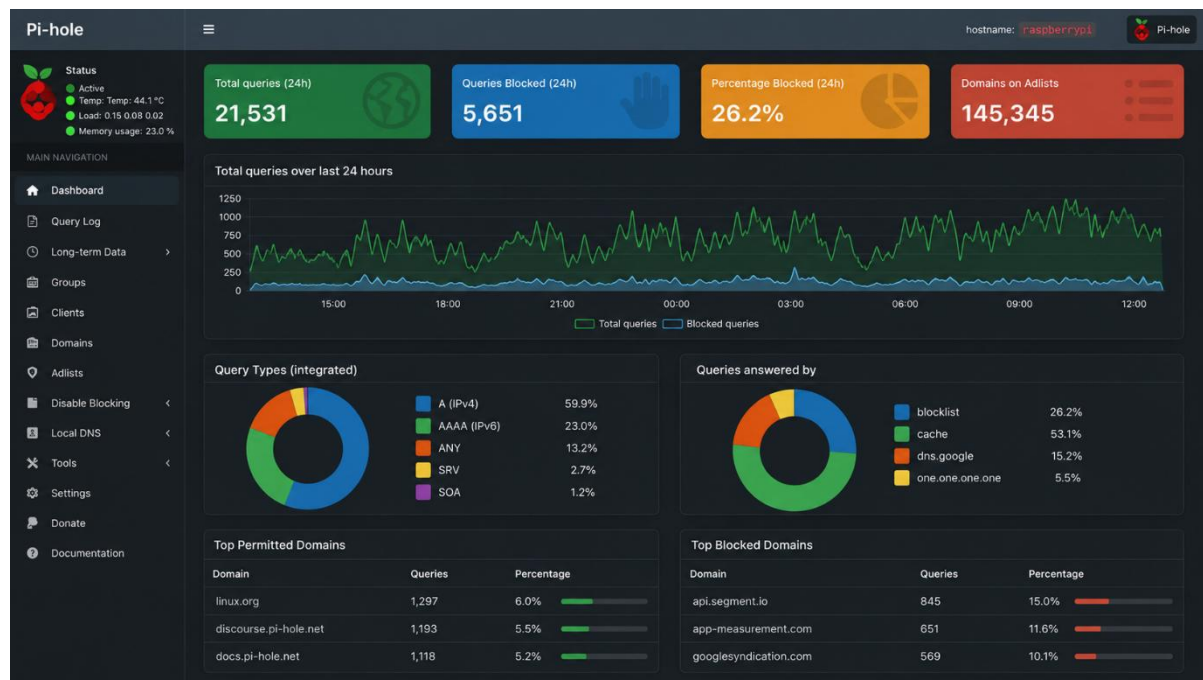
Su función principal fue:

- Resolver dominios internos
- Centralizar consultas DNS
- Filtrar publicidad
- Facilitar acceso a servicios mediante nombres personalizados

Gracias a Pi-hole fue posible acceder a los servicios utilizando dominios internos como:

- grafana.local
- nextcloud.local
- glpi.local
- portainer.local

Esto permitió crear una infraestructura más organizada y profesional.

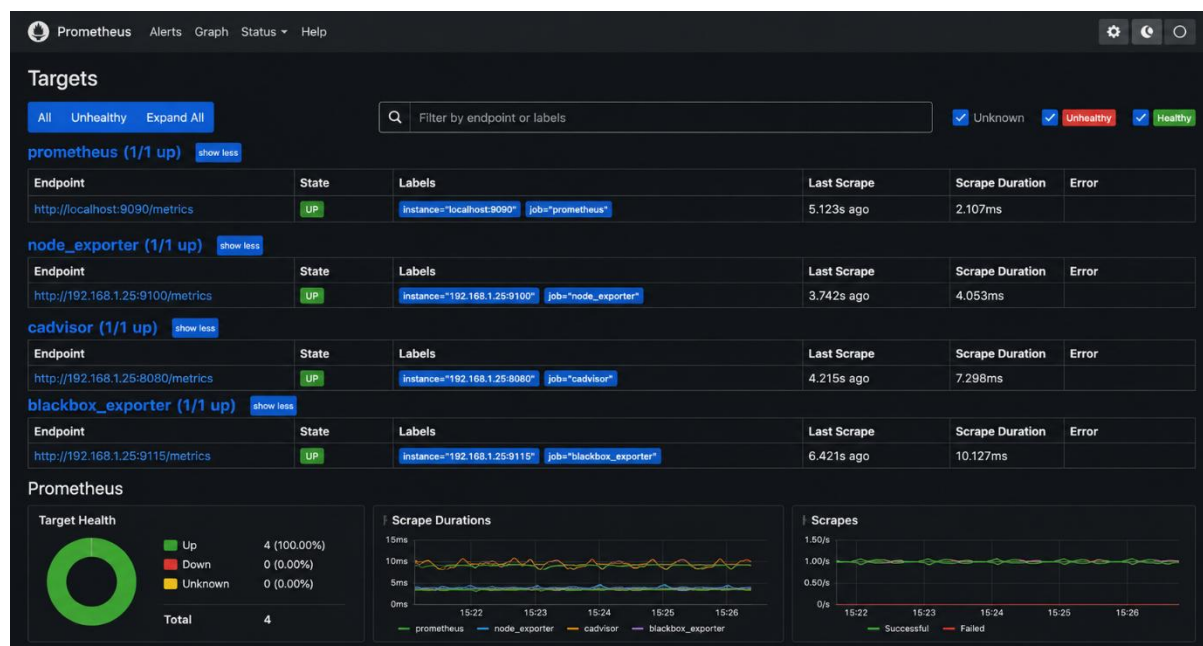


Implementación de servicios de monitorización

Se desplegó un sistema completo de monitorización basado en varias herramientas especializadas.

Prometheus

Se utilizó para recopilar métricas del sistema y de los contenedores.



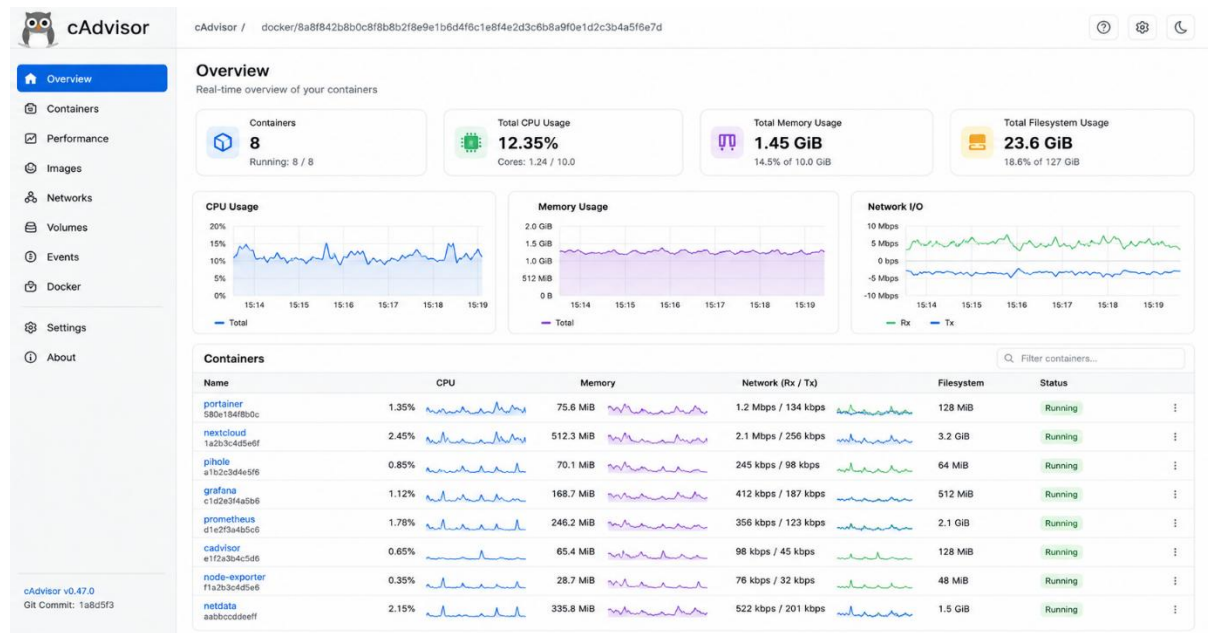
Grafana

Se utilizó para visualizar métricas mediante dashboards interactivos.



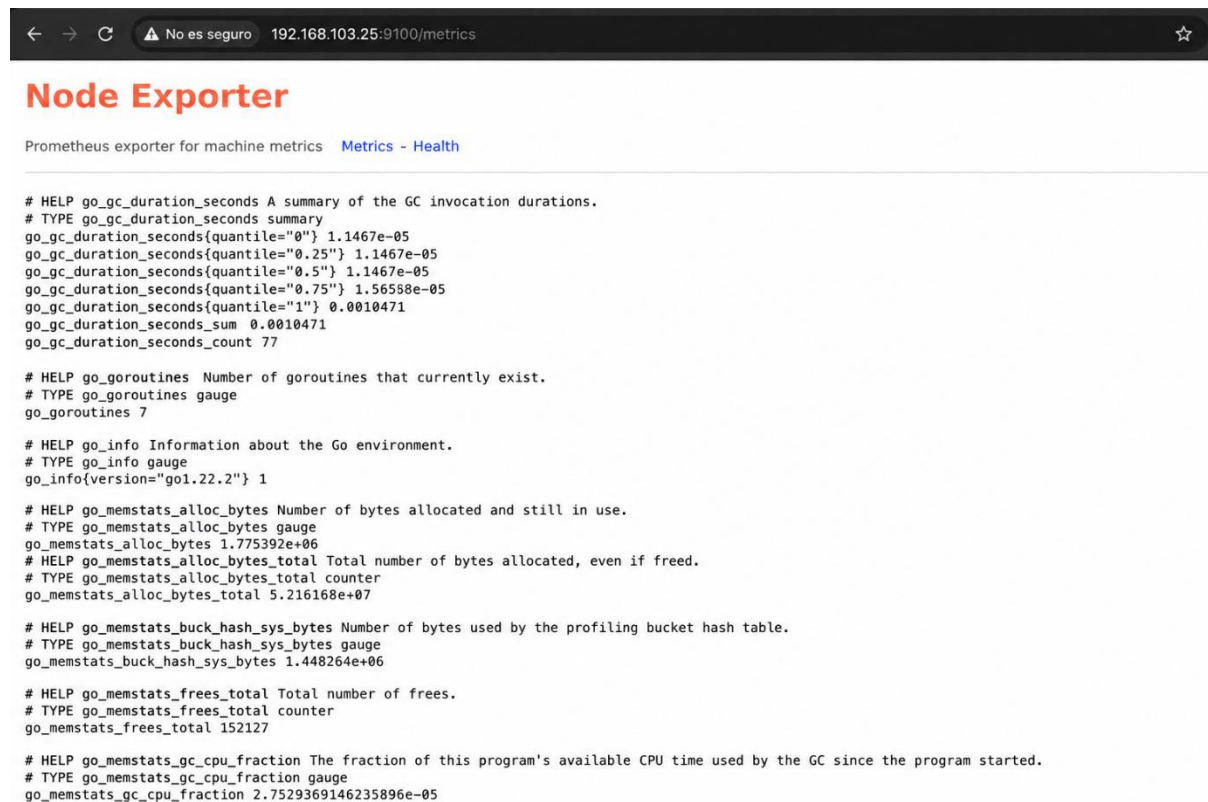
cAdvisor

Permitió monitorizar el consumo de recursos de los contenedores Docker.



Node Exporter

Permitió obtener métricas del sistema operativo Linux.



```
# HELP go_gc_duration_seconds A summary of the GC invocation durations.
# TYPE go_gc_duration_seconds summary
go_gc_duration_seconds{quantile="0"} 1.1467e-05
go_gc_duration_seconds{quantile="0.25"} 1.1467e-05
go_gc_duration_seconds{quantile="0.5"} 1.1467e-05
go_gc_duration_seconds{quantile="0.75"} 1.5658e-05
go_gc_duration_seconds{quantile="1"} 0.0010471
go_gc_duration_seconds_sum 0.0010471
go_gc_duration_seconds_count 77

# HELP go_goroutines Number of goroutines that currently exist.
# TYPE go_goroutines gauge
go_goroutines 7

# HELP go_info Information about the Go environment.
# TYPE go_info gauge
go_info{version="go1.22.2"} 1

# HELP go_memstats_alloc_bytes Number of bytes allocated and still in use.
# TYPE go_memstats_alloc_bytes gauge
go_memstats_alloc_bytes 1.775392e+06
# HELP go_memstats_alloc_bytes_total Total number of bytes allocated, even if freed.
# TYPE go_memstats_alloc_bytes_total counter
go_memstats_alloc_bytes_total 5.216168e+07

# HELP go_memstats_buck_hash_sys_bytes Number of bytes used by the profiling bucket hash table.
# TYPE go_memstats_buck_hash_sys_bytes gauge
go_memstats_buck_hash_sys_bytes 1.448264e+06

# HELP go_memstats_frees_total Total number of frees.
# TYPE go_memstats_frees_total counter
go_memstats_frees_total 152127

# HELP go_memstats_gc_cpu_fraction The fraction of this program's available CPU time used by the GC since the program started.
# TYPE go_memstats_gc_cpu_fraction gauge
go_memstats_gc_cpu_fraction 2.7529369146235896e-05
```

Netdata

Se implementó como herramienta de monitorización en tiempo real del servidor.

Todo el sistema de monitorización permitió supervisar:

- CPU
- Memoria RAM
- Disco
- Red
- Rendimiento Docker
- Estado de servicios

La siguiente imagen muestra el panel principal de Netdata, utilizado para supervisar en tiempo real el estado general de la Raspberry Pi.



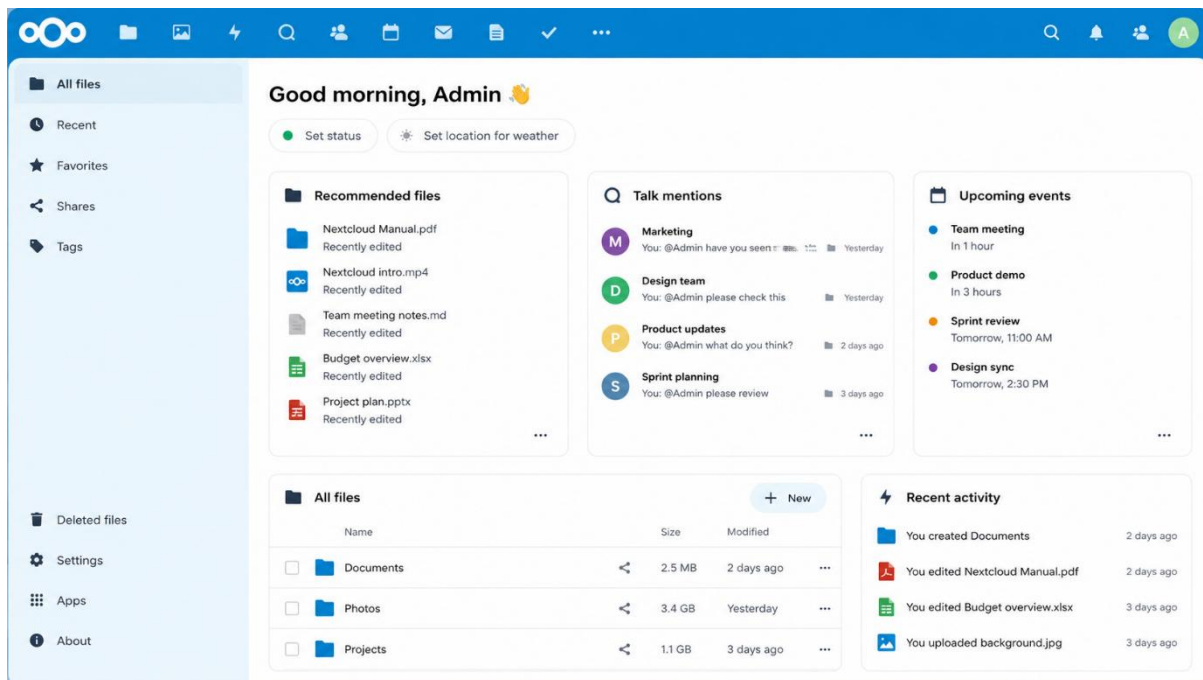
Implementación de Nextcloud

Se desplegó Nextcloud como plataforma de almacenamiento privado y gestión documental.

Sus funciones principales fueron:

- Almacenamiento de archivos
- Compartición de documentos
- Gestión de proyectos
- Acceso web centralizado
- Sincronización de archivos

Esto permitió disponer de una nube privada dentro de la red local.

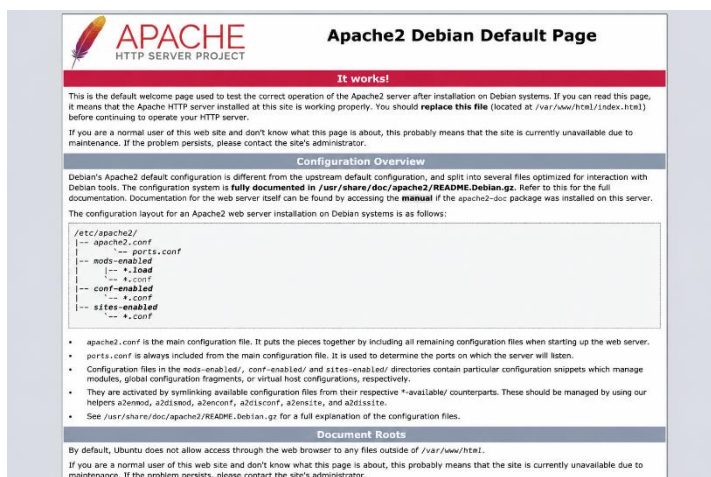


Implementación de servidores web Apache

Se desplegaron múltiples contenedores Apache con fines de pruebas y simulación de entornos web.

Estos servidores fueron utilizados para:

- Prácticas web
- Simulación de clientes
- Publicación de contenido
- Entornos de pruebas



2.4 Objetivos de Aprendizaje

Además de los objetivos técnicos, el proyecto tuvo una finalidad formativa orientada al aprendizaje práctico de tecnologías reales utilizadas en entornos profesionales.

Durante el desarrollo del proyecto se trabajaron conocimientos relacionados con:

- Administración Linux
- Redes locales
- Docker y contenedores
- Virtualización ligera
- Servicios DNS
- Monitorización de sistemas
- Gestión de servidores
- Administración web
- Resolución de errores
- Seguridad básica de infraestructura

2.5 Tecnologías Utilizadas

Tecnología	Función Principal
Raspberry Pi OS	Sistema operativo del servidor
Docker	Virtualización mediante contenedores
Portainer	Gestión visual de Docker
Pi-hole	Servidor DNS interno
Nextcloud	Nube privada y almacenamiento
Prometheus	Recolección de métricas
Grafana	Visualización de métricas
Netdata	Monitorización en tiempo real
cAdvisor	Métricas de contenedores Docker

Node Exporter	Métricas del sistema Linux
Apache HTTPD	Servidores web

3. Hardware Utilizado

Equipamiento principal

- Raspberry Pi 4 con 4 GB de RAM
- Tarjeta microSD
- Switch de red
- Cableado Ethernet CAT8
- Monitor HP EliteDisplay
- Clientes Windows
- Servidor Proxmox
- Teclado y ratón USB

Raspberry Pi 4

La Raspberry Pi 4 fue utilizada como servidor principal del proyecto.

En ella se instaló Raspberry Pi OS y posteriormente Docker junto con todos los servicios desplegados mediante contenedores.

Funciones principales

- Ejecución de contenedores Docker
- Gestión de servicios web
- Monitorización del sistema
- Almacenamiento privado
- Servidor DNS interno

Las siguientes imágenes muestran el hardware principal utilizado para el despliegue de la infraestructura. La Raspberry Pi 4 fue utilizada como servidor principal para alojar todos los servicios.



Tarjeta microSD

La tarjeta microSD fue utilizada como unidad de almacenamiento principal.

Uso dentro del proyecto

- Instalación del sistema operativo
- Almacenamiento de configuraciones
- Persistencia de contenedores
- Bases de datos y archivos



Switch de Red

El switch permitió interconectar todos los dispositivos de la infraestructura.

Equipos conectados

- Raspberry Pi
- Clientes Windows
- Servidor Proxmox
- Equipos de prácticas



Cableado Ethernet CAT8

Se utilizó cableado CAT8 para garantizar una conexión estable y rápida entre todos los dispositivos de la red.

Ventajas

- Mayor velocidad
- Menor interferencia
- Mejor estabilidad de conexión

Clientes Windows

Los clientes Windows fueron utilizados para acceder y probar los distintos servicios desplegados.

Servicios utilizados

- Portainer
- Nextcloud
- Grafana
- Netdata
- Prometheus

Monitor y periféricos

Se utilizaron un monitor HP EliteDisplay, teclado USB y ratón USB para la administración local del sistema.

4. Arquitectura General

La arquitectura del proyecto fue diseñada para centralizar múltiples servicios en una única Raspberry Pi utilizando contenedores Docker.

Todos los servicios funcionan de forma independiente dentro de contenedores, permitiendo una gestión modular, sencilla y escalable.

Función principal de la Raspberry Pi

La Raspberry Pi actúa como núcleo principal de la infraestructura.

Servicios principales alojados

- Host principal Docker
- Servidor DNS interno mediante Pi hole
- Plataforma de monitorización
- Servidor web
- Sistema de almacenamiento privado
- Gestor de contenedores

Estructura de la infraestructura

La infraestructura está compuesta por varios servicios desplegados en contenedores Docker conectados dentro de la misma red local.

Componentes desplegados

Servicio	Función
Portainer	Administración visual de contenedores Docker
Pi hole	Servidor DNS y bloqueo de publicidad
Nextcloud	Plataforma de almacenamiento privado
Prometheus	Recolección de métricas
Grafana	Visualización y dashboards
Netdata	Monitorización en tiempo real
cAdvisor	Monitorización de contenedores
Node Exporter	Métricas del sistema Linux

Funcionamiento general

El sistema funciona mediante una arquitectura basada en servicios independientes.

Cada contenedor ejecuta una función concreta:

- Prometheus recopila métricas del sistema y contenedores.
- Grafana muestra gráficos y paneles de monitorización.
- cAdvisor obtiene estadísticas de Docker.
- Node Exporter recopila información del sistema operativo.
- Netdata monitoriza recursos en tiempo real.
- Pi hole gestiona las peticiones DNS de la red.
- Nextcloud proporciona almacenamiento web privado.
- Portainer centraliza la administración de todos los contenedores.

Red local

Todos los servicios fueron integrados dentro de la misma red local mediante direcciones IP privadas.

La Raspberry Pi se conectó mediante Ethernet para mejorar:

- Estabilidad
- Velocidad
- Disponibilidad de servicios
- Acceso simultáneo desde clientes

Gestión mediante Docker

Docker permitió desplegar todos los servicios de forma aislada y organizada.

Ventajas obtenidas

- Facilidad de despliegue
- Separación de servicios
- Portabilidad
- Administración centralizada
- Reinicio independiente de servicios
- Escalabilidad futura

Administración centralizada

La administración de toda la infraestructura se realizó desde Portainer mediante interfaz web.

Desde esta plataforma fue posible:

- Crear contenedores
- Reiniciar servicios
- Visualizar estados
- Gestionar imágenes Docker
- Supervisar recursos
- Configurar redes y volúmenes

5. Instalación del Sistema Operativo

Instalación de Raspberry Pi OS

Para el desarrollo del proyecto se utilizó Raspberry Pi OS como sistema operativo principal de la infraestructura.

La instalación se realizó utilizando la herramienta Raspberry Pi Imager, permitiendo grabar el sistema operativo directamente sobre una tarjeta microSD.

Descarga del sistema operativo

Se descargó la versión oficial de Raspberry Pi OS desde la página oficial de Raspberry Pi.

Herramientas utilizadas

- Raspberry Pi Imager
- Tarjeta microSD
- Lector de tarjetas SD



Grabación de la imagen del sistema

La imagen del sistema operativo fue grabada en la tarjeta microSD mediante Raspberry Pi Imager.

Proceso realizado

1. Selección del sistema operativo.
2. Selección de la tarjeta microSD.
3. Configuración inicial avanzada.
4. Escritura de la imagen.
5. Verificación automática.

Configuración inicial

Antes del primer arranque se configuraron varios parámetros esenciales del sistema.

Parámetros configurados

- Usuario principal
- Contraseña
- Acceso SSH
- Nombre del equipo
- Zona horaria
- Idioma del sistema
- Configuración de red Ethernet

Primer arranque

Una vez insertada la tarjeta microSD en la Raspberry Pi, el sistema arrancó correctamente y permitió acceder al entorno de escritorio y terminal Linux.

La Raspberry Pi quedó conectada a la red local mediante cable Ethernet para garantizar estabilidad y conectividad permanente.

Verificación de conectividad

Tras el arranque se verificó la conectividad de red y la asignación de dirección IP.

Comandos utilizados

```
hostname -I
```

```
ping google.com
```

Resultado obtenido

- Conectividad correcta a Internet
- Comunicación dentro de la red local
- Dirección IP asignada correctamente

Actualización del sistema

Después de la instalación inicial se realizó una actualización completa del sistema operativo y paquetes instalados.

Comandos utilizados

```
sudo apt update
```

```
sudo apt upgrade -y
```

Objetivos de la actualización

- Instalar paquetes recientes
- Corregir vulnerabilidades
- Mejorar estabilidad del sistema
- Actualizar dependencias necesarias para Docker

Configuración de acceso remoto

Se habilitó el servicio SSH para administrar la Raspberry Pi remotamente desde otros equipos de la red.

Ventajas

- Administración remota
- Acceso mediante terminal
- Gestión sin monitor físico
- Mayor comodidad de administración

6. Instalación de Docker

Docker se utilizó como plataforma principal para desplegar y gestionar todos los servicios del proyecto mediante contenedores.

Gracias a Docker fue posible ejecutar aplicaciones independientes dentro de la misma Raspberry Pi de forma organizada y eficiente.

Objetivo de Docker dentro del proyecto

Docker permitió:

- Ejecutar servicios aislados
- Simplificar instalaciones
- Centralizar aplicaciones
- Gestionar contenedores fácilmente
- Reducir conflictos entre servicios
- Facilitar la monitorización del sistema

Todos los servicios del proyecto fueron desplegados mediante contenedores Docker.

Instalación de Docker

La instalación se realizó utilizando el instalador oficial de Docker para Linux.

Comando utilizado

```
curl -fsSL https://get.docker.com | sh
```

Este comando descarga e instala automáticamente:

- Docker Engine
- Docker CLI

- Dependencias necesarias
- Servicios del sistema Docker

Verificación de instalación

Una vez finalizada la instalación, se verificó que Docker estuviese correctamente instalado y funcionando.

Comando utilizado

```
docker --version
```

Resultado esperado

El sistema devuelve la versión instalada de Docker.

Ejemplo:

Docker version 28.x.x

Configuración de permisos

Para evitar utilizar sudo constantemente, el usuario principal fue añadido al grupo docker.

Comando utilizado

```
sudo usermod -aG docker $USER
```

Posteriormente se reinició sesión para aplicar los cambios correctamente.

Verificación del servicio Docker

Se comprobó que el servicio Docker estuviese activo y funcionando correctamente.

Comando utilizado

```
sudo systemctl status docker
```

Resultado esperado

- Servicio activo
- Docker ejecutándose correctamente
- Inicio automático habilitado

Funcionamiento de Docker

Docker trabaja mediante:

- Imágenes
- Contenedores
- Redes virtuales
- Volúmenes persistentes

Cada servicio del proyecto se ejecutó dentro de su propio contenedor independiente.

Ventajas obtenidas

Organización

Cada servicio funciona separado del resto.

Seguridad

Los contenedores trabajan de forma aislada.

Portabilidad

Los servicios pueden migrarse fácilmente.

Escalabilidad

Es posible añadir nuevos servicios rápidamente.

Administración sencilla

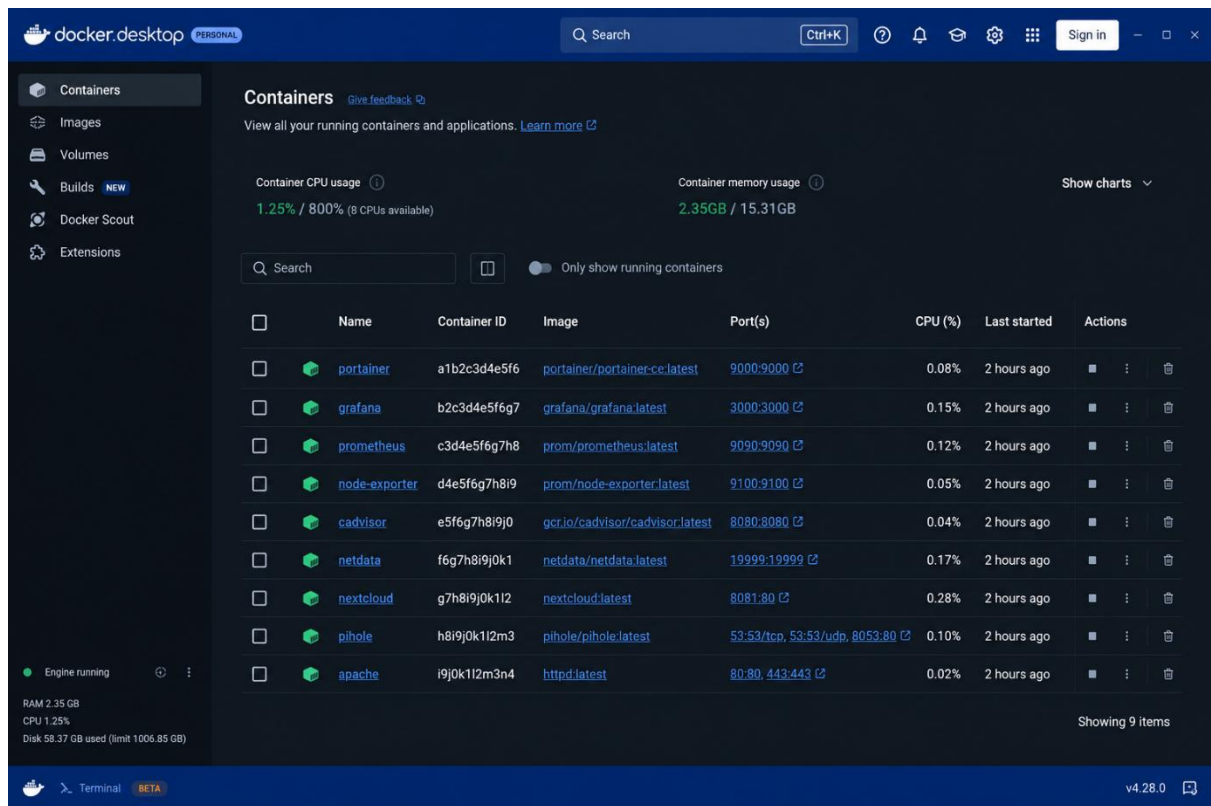
Todos los contenedores pueden gestionarse desde Portainer.

Servicios desplegados posteriormente

Tras instalar Docker se preparó el sistema para desplegar:

- Portainer
- Pi hole
- Nextcloud
- Prometheus
- Grafana
- Netdata
- cAdvisor
- Node Exporter

La siguiente imagen muestra el entorno Docker utilizado durante el proyecto. En él se pueden observar todos los contenedores desplegados en la Raspberry Pi.



7. Instalación de Portainer

Portainer se utilizó como plataforma de administración visual para gestionar todos los contenedores Docker desde una interfaz web.

Gracias a Portainer fue posible controlar servicios, imágenes, redes y volúmenes de forma centralizada sin necesidad de utilizar únicamente comandos por terminal.

Objetivo de Portainer

Portainer permitió:

- Gestionar contenedores Docker
- Visualizar el estado de los servicios
- Reiniciar y detener contenedores
- Administrar imágenes Docker
- Supervisar recursos

- Gestionar redes y volúmenes
- Facilitar la administración del sistema

Creación del volumen persistente

Antes del despliegue del contenedor se creó un volumen persistente para almacenar la configuración de Portainer.

Comando utilizado

```
docker volume create portainer_data
```

Función del volumen

El volumen permite conservar:

- Usuarios
- Configuraciones
- Preferencias
- Datos administrativos

Aunque el contenedor sea reiniciado o actualizado.

Despliegue del contenedor

El contenedor Portainer fue desplegado utilizando Docker.

Comando utilizado

```
docker run -d \  
-p 8000:8000 \  
-p 9443:9443 \  
--name portainer \  
--restart=always \  
-v /var/run/docker.sock:/var/run/docker.sock \  
-v portainer_data:/data \  
portainer/portainer-ce:latest
```

Explicación de parámetros

Parámetro	Función
-d	Ejecuta el contenedor en segundo plano
-p 8000:8000	Puerto interno de Portainer
-p 9443:9443	Acceso web HTTPS
--name portainer	Nombre del contenedor
--restart=always	Reinicio automático
/var/run/docker.sock	Control de Docker desde Portainer
portainer_data:/data	Persistencia de datos

Acceso web

Una vez desplegado el contenedor, se accedió a Portainer desde navegador web.

Dirección utilizada

<https://192.168.103.25:9443>

Configuración inicial

Durante el primer acceso se configuró:

- Usuario administrador
- Contraseña segura
- Entorno Docker local

Posteriormente Portainer quedó conectado al motor Docker de la Raspberry Pi.

Gestión de contenedores

Desde Portainer se administraron todos los servicios del proyecto.

Contenedores gestionados

- Pi hole
- Nextcloud
- Prometheus
- Grafana
- Netdata
- cAdvisor
- Node Exporter

Funcionalidades utilizadas

Administración de contenedores

- Inicio y parada de servicios
- Reinicio de contenedores
- Visualización de estados

Gestión de imágenes

- Descarga de imágenes Docker
- Eliminación de imágenes innecesarias

Supervisión

- Consumo de CPU
- Uso de memoria
- Redes activas
- Estado de contenedores

Gestión de almacenamiento

- Volúmenes Docker
- Persistencia de datos

Seguridad

Portainer se configuró mediante HTTPS utilizando el puerto 9443 para mejorar la seguridad de acceso web.

8. Configuración de Pi hole

Objetivo

Pi hole fue implementado como servidor DNS interno de la infraestructura para gestionar la resolución de nombres dentro de la red local y mejorar el control del tráfico DNS.

Además de resolver nombres, Pi hole permitió bloquear publicidad y dominios innecesarios dentro de la red.

Funciones principales de Pi hole

Resolución DNS interna

Permite transformar nombres de dominio en direcciones IP dentro de la red local.

Bloqueo de publicidad

Filtra automáticamente dominios publicitarios y rastreadores.

Gestión centralizada DNS

Todos los equipos de la red utilizan el mismo servidor DNS.

Mejora de privacidad

Reduce conexiones hacia dominios de seguimiento.

Despliegue mediante Docker

Pi hole fue desplegado como contenedor Docker dentro de la Raspberry Pi.

La instalación se realizó desde Portainer utilizando una imagen oficial compatible con ARM64.

Puertos utilizados

Puerto	Función
53 TCP/UDP	Servicio DNS
80	Panel web
8099	Acceso web personalizado

Acceso web

El panel de administración quedó accesible desde navegador web.

Dirección utilizada

<http://192.168.103.25:8099/admin>

Configuración DNS

Se configuraron servidores DNS externos para reenviar consultas no locales.

DNS upstream configurados

- Google DNS
- Cloudflare DNS

Direcciones utilizadas

Servicio	Dirección
Google DNS	8.8.8.8
Cloudflare DNS	1.1.1.1

Integración en la red local

Los clientes de la red fueron configurados para utilizar la Raspberry Pi como servidor DNS principal.

Resultado obtenido

- Resolución DNS centralizada
- Control del tráfico DNS
- Bloqueo de publicidad
- Gestión simplificada

Funcionamiento interno

El flujo de funcionamiento quedó estructurado de la siguiente forma:

1. El cliente realiza una petición DNS.
2. Pi hole recibe la solicitud.
3. Se verifica si el dominio está bloqueado.
4. Si está permitido, la petición se reenvía al DNS externo.
5. Pi hole devuelve la respuesta al cliente.

Ventajas obtenidas

Mayor control de red

Todas las consultas DNS pasan por un único punto.

Reducción de publicidad

Se bloquean dominios publicitarios automáticamente.

Mejor rendimiento

Las consultas DNS frecuentes quedan cacheadas.

Administración sencilla

Toda la configuración puede gestionarse desde interfaz web.

Administración desde Portainer

El contenedor Pi hole pudo administrarse desde Portainer.

Acciones disponibles

- Reinicio del servicio
- Supervisión del estado
- Consulta de logs
- Gestión de red
- Configuración de volúmenes

9. Creación de DNS Locales

Objetivo

Se configuraron registros DNS locales en Pi hole para facilitar el acceso a los servicios de la infraestructura mediante nombres personalizados en lugar de utilizar direcciones IP.

Esto permitió acceder a los distintos servicios desde navegador web de forma más sencilla y organizada.

Configuración de DNS locales

La configuración se realizó desde el panel de administración de Pi hole.

Ruta utilizada

Settings → Local DNS Records

Desde esta sección se añadieron manualmente los dominios personalizados junto a sus direcciones IP correspondientes.

Funcionamiento de los registros DNS

Cada dominio configurado apunta directamente a la dirección IP local de la Raspberry Pi.

Cuando un cliente realiza una consulta DNS:

1. Pi hole recibe la petición.
2. Busca el dominio dentro de los registros locales.
3. Devuelve automáticamente la dirección IP asociada.

Dominios configurados

Durante el proyecto se configuraron varios dominios personalizados para acceder rápidamente a los distintos servicios.

Ejemplos utilizados

Dominio	Función
asir1.local	Servidor web ASIR
asir2.local	Segundo entorno web
grafana.local	Acceso a Grafana
masterciber1.local	Entorno de prácticas
docker.local	Administración Docker

Dirección IP utilizada

Todos los dominios locales apuntaban a la dirección IP de la Raspberry Pi dentro de la red local.

Dirección configurada

192.168.103.34

Ventajas de utilizar DNS locales

Acceso simplificado

No es necesario memorizar direcciones IP.

Organización de servicios

Cada aplicación tiene un nombre identificativo.

Mejor administración

Permite gestionar servicios de forma más profesional.

Escalabilidad

Es posible añadir nuevos dominios fácilmente.

Integración con los clientes

Los clientes Windows configurados en la red utilizaron Pi hole como DNS principal.

Gracias a esto, todos los equipos podían resolver automáticamente:

- grafana.local
- docker.local
- asir1.local
- Otros dominios internos

Sin necesidad de editar manualmente archivos hosts.

Verificación de funcionamiento

Se verificó la resolución DNS utilizando navegador web y herramientas de red.

Comandos utilizados

```
ping grafana.local
```

```
nslookup docker.local
```

10. Configuración de Clientes Windows

Objetivo

Los clientes Windows fueron configurados para utilizar la Raspberry Pi como servidor DNS principal dentro de la red local.

Gracias a esta configuración, todos los equipos pudieron acceder a los servicios utilizando nombres personalizados en lugar de direcciones IP.

Configuración manual del DNS

La configuración se realizó manualmente desde las propiedades de red de Windows.

Ruta utilizada

Panel de control → Centro de redes → Cambiar configuración del adaptador

Posteriormente:

Propiedades → IPv4 → Usar las siguientes direcciones DNS

DNS configurado

Se estableció la dirección IP de la Raspberry Pi como servidor DNS principal.

Dirección configurada

192.168.103.25

Funcionamiento

Una vez aplicada la configuración:

1. El cliente Windows envía las consultas DNS a la Raspberry Pi.
2. Pi lo resuelve los dominios locales.
3. El cliente obtiene automáticamente la dirección IP correcta.

Acceso a servicios mediante dominios locales

Tras la configuración DNS, los equipos pudieron acceder a los servicios utilizando nombres personalizados.

Ejemplos funcionales

`http://grafana.local`

`http://masterciber1.local`

`http://docker.local`

Verificación de conectividad

Se realizaron pruebas desde los clientes Windows para comprobar el correcto funcionamiento de la resolución DNS.

Herramientas utilizadas

- Navegador web
- Ping
- Nslookup

Ejemplos de comandos

ping grafana.local

nslookup docker.local

Ventajas obtenidas

Acceso más sencillo

No es necesario recordar direcciones IP.

Resolución centralizada

Todos los clientes utilizan el mismo servidor DNS.

Organización de servicios

Cada aplicación dispone de un dominio identificativo.

Administración profesional

La infraestructura funciona de forma similar a entornos empresariales reales.

Integración dentro de la red

Todos los clientes Windows quedaron integrados dentro de la misma infraestructura de red gestionada por la Raspberry Pi.

Esto permitió:

- Resolución DNS local
- Acceso a servicios web
- Comunicación entre equipos
- Gestión centralizada

11. Monitorización

Objetivo

La infraestructura fue equipada con varias herramientas de monitorización para supervisar el estado de la Raspberry Pi, los contenedores Docker y los servicios desplegados.

La monitorización permitió controlar:

- Uso de CPU
- Consumo de memoria RAM
- Actividad de red
- Estado de contenedores
- Rendimiento del sistema
- Servicios activos

Grafana

Función principal

Grafana se utilizó como plataforma de visualización para representar métricas mediante paneles gráficos y dashboards interactivos.

La herramienta permitió centralizar toda la información obtenida desde Prometheus.

Funciones implementadas

Visualización de métricas

Grafana permitió representar:

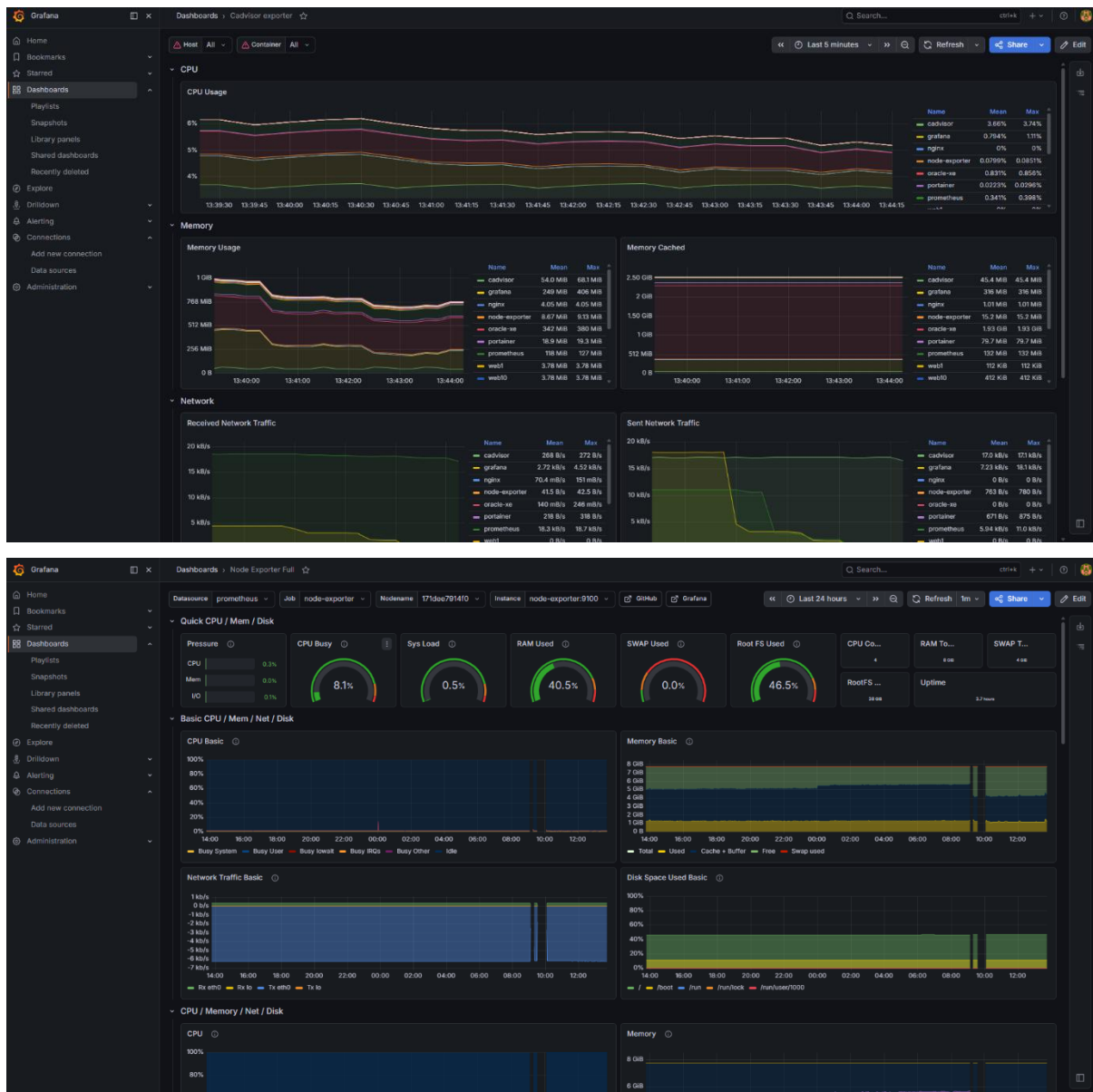
- CPU
- RAM
- Red
- Contenedores Docker
- Estado del sistema

Dashboards personalizados

Se crearon paneles para visualizar el estado general de la infraestructura.

Supervisión en tiempo real

Los datos se actualizaban automáticamente mostrando el estado actual del sistema.



Integración con Prometheus

Grafana utilizó Prometheus como fuente de datos principal.

Flujo de funcionamiento

1. Node Exporter y cAdvisor generan métricas.
2. Prometheus recopila la información.
3. Grafana obtiene los datos desde Prometheus.
4. Los dashboards muestran gráficos y estadísticas.

Acceso web

Grafana quedó accesible desde navegador web.

Dirección utilizada

<http://192.168.103.25:3000>

Funciones utilizadas en Grafana

Dashboards

- Monitorización general
- Estado del sistema
- Recursos Docker

Métricas visualizadas

- Uso de CPU
- Memoria RAM
- Uso de disco
- Red
- Estado de servicios

Netdata

Función principal

Netdata se utilizó como sistema de monitorización en tiempo real para supervisar el rendimiento de la Raspberry Pi y los contenedores Docker.

A diferencia de Grafana, Netdata muestra métricas instantáneas directamente desde el sistema.

Información monitorizada

CPU

Uso de procesador en tiempo real.

RAM

Consumo de memoria del sistema.

Red

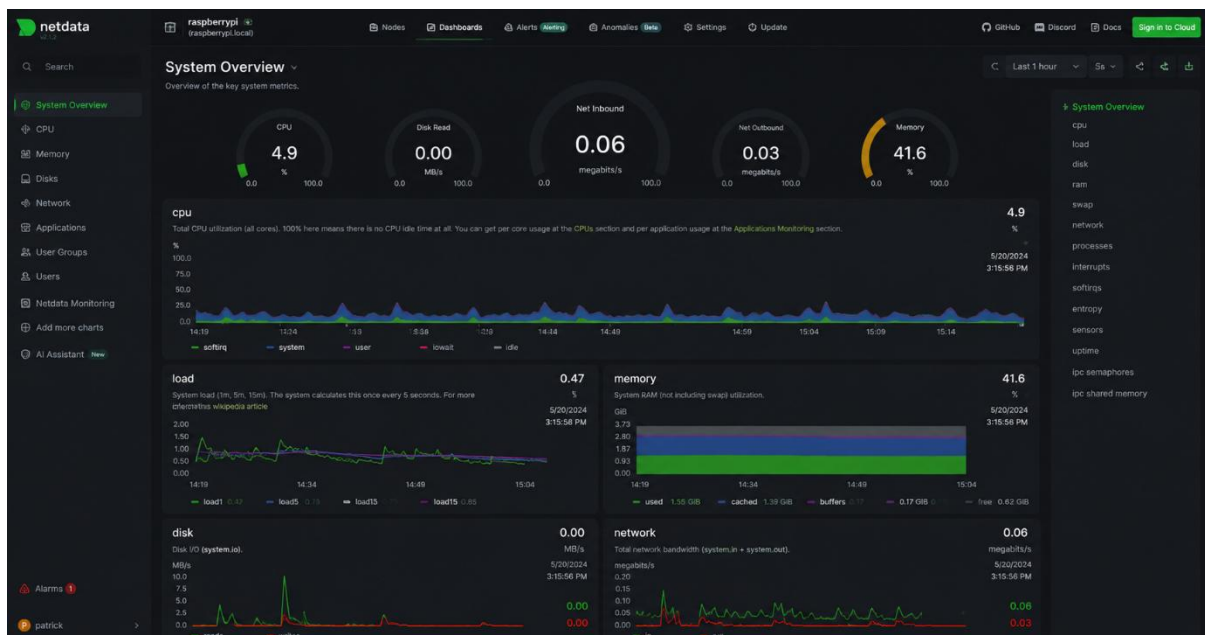
Tráfico de red y actividad Ethernet.

Docker

Estado y consumo de recursos de los contenedores.

Disco

Uso de almacenamiento y actividad del sistema.



Características principales

Tiempo real

Actualización constante de métricas.

Interfaz gráfica

Panel web visual e interactivo.

Bajo consumo

Optimizado para dispositivos ligeros como Raspberry Pi.

Supervisión automática

Detección automática de servicios y contenedores.

Acceso web

Netdata quedó accesible desde navegador web.

Dirección utilizada

<http://192.168.103.25:19999>

cAdvisor

Función principal

cAdvisor se utilizó para recopilar estadísticas específicas de los contenedores Docker.

Información monitorizada

- Uso de CPU
- Memoria por contenedor
- Red Docker
- Consumo de disco
- Estado de contenedores

Integración con Prometheus

Prometheus recopilaba automáticamente las métricas generadas por cAdvisor.

Acceso web

Dirección utilizada

<http://192.168.103.25:8089>

Node Exporter

Función principal

Node Exporter permitió recopilar métricas internas del sistema operativo Linux.

Métricas recopiladas

- CPU
- RAM
- Disco
- Temperatura
- Red
- Procesos activos

Integración con Prometheus

Prometheus utilizó Node Exporter para obtener información completa del sistema Linux de la Raspberry Pi.

Acceso web

Dirección utilizada

<http://192.168.103.25:9100>

Prometheus

Función principal

Prometheus se utilizó como servidor principal de recopilación de métricas.

Funcionamiento

Prometheus obtiene información desde:

- Node Exporter
- cAdvisor

Posteriormente almacena las métricas para ser visualizadas desde Grafana.

Acceso web

Dirección utilizada

<http://192.168.103.25:9090>

Resultado final

La infraestructura quedó completamente monitorizada mediante múltiples herramientas integradas entre sí.

Esto permitió:

- Supervisión en tiempo real
- Visualización gráfica avanzada
- Monitorización de Docker
- Control de recursos del sistema
- Gestión centralizada de métricas

12. Seguridad Implementada

Objetivo

Se aplicaron distintas medidas de seguridad para proteger la infraestructura desplegada en la Raspberry Pi y limitar el acceso únicamente a los dispositivos autorizados dentro de la red local.

La seguridad se centró principalmente en:

- Aislamiento de servicios
- Control de acceso
- DNS interno privado
- Gestión centralizada
- Segmentación mediante contenedores

DNS local privado

Pi hole fue configurado como servidor DNS interno para evitar depender completamente de servicios externos.

Ventajas obtenidas

- Resolución DNS local
- Mayor control sobre la red
- Gestión centralizada de dominios
- Acceso mediante nombres personalizados
- Reducción de tráfico innecesario

Acceso únicamente dentro de la red local

Todos los servicios fueron configurados para funcionar únicamente dentro de la red local privada.

Resultado

Los servicios no quedaron expuestos directamente a Internet.

Servicios protegidos

- Portainer
- Grafana
- Nextcloud
- Netdata
- Prometheus
- cAdvisor
- Node Exporter

Gestión centralizada

La administración de toda la infraestructura se realizó desde Portainer.

Esto permitió:

- Supervisar servicios
- Gestionar contenedores
- Reiniciar aplicaciones

- Consultar estados
- Administrar recursos

Desde una única plataforma centralizada.

Contenedores aislados

Cada servicio fue desplegado dentro de su propio contenedor Docker independiente.

Ventajas de aislamiento

- Separación de servicios
- Menor impacto ante errores
- Mayor estabilidad
- Mejor organización
- Control independiente de recursos

Uso de Docker

Docker aportó una capa adicional de organización y aislamiento dentro de la infraestructura.

Beneficios obtenidos

- Servicios independientes
- Reinicio individual de contenedores
- Facilidad de mantenimiento
- Despliegue rápido
- Escalabilidad

Seguridad de acceso web

Algunos servicios fueron configurados mediante HTTPS y puertos específicos para controlar el acceso web.

Ejemplo

Portainer se configuró utilizando:

<https://192.168.103.25:9443>

Seguridad de usuarios

Se configuraron credenciales administrativas para proteger el acceso a:

- Portainer
- Grafana
- Nextcloud
- Pi hole

Supervisión del sistema

Las herramientas de monitorización permitieron detectar:

- Consumo excesivo de recursos
- Estado de servicios
- Fallos en contenedores
- Actividad del sistema

Esto ayudó a mantener la estabilidad general de la infraestructura.

Resultado final

La infraestructura quedó protegida mediante una combinación de:

- DNS privado
- Red local aislada
- Contenedores independientes
- Gestión centralizada
- Control de acceso web

Todo ello permitió disponer de un entorno estable, organizado y seguro para servicios Docker desplegados en Raspberry Pi.

13. Problemas Encontrados y Soluciones

Durante el desarrollo del proyecto aparecieron distintos problemas relacionados con Docker, compatibilidad ARM64, red y configuración de servicios.

Cada incidencia fue solucionada mediante pruebas, cambios de configuración y revisión de contenedores.

Problema 1. Pi hole command not found

Descripción

Inicialmente se intentó ejecutar comandos de Pi hole directamente desde el sistema operativo, provocando errores porque Pi hole estaba desplegado dentro de Docker.

Error detectado

```
pihole: command not found
```

Causa

Pi hole funcionaba dentro de un contenedor Docker y no como instalación nativa del sistema operativo.

Solución aplicada

Se comenzó a gestionar Pi hole directamente desde:

- Portainer
- Docker
- Interfaz web de administración

También se utilizaron comandos Docker para acceder al contenedor.

Resultado

Pi hole quedó funcionando correctamente dentro del entorno Docker.

Problema 2. El DNS no resolvía dominios locales

Descripción

Los equipos Windows no podían resolver dominios personalizados como:

- grafana.local
- docker.local

Causa

Los clientes seguían utilizando DNS externos en lugar del DNS interno configurado en Pi hole.

Solución aplicada

Se configuró manualmente el DNS en IPv4 utilizando la IP de la Raspberry Pi.

Configuración aplicada

192.168.103.25

Resultado

Los clientes pudieron resolver correctamente todos los dominios locales.

Problema 3. Error ARM64 en contenedores

Descripción

Algunos contenedores no funcionaban correctamente debido a incompatibilidad con arquitectura ARM64 de Raspberry Pi.

Error detectado

exec format error

Causa

La imagen Docker utilizada estaba preparada para arquitectura x86 y no para ARM64.

Solución aplicada

Se sustituyeron las imágenes incompatibles por versiones compatibles con Raspberry Pi ARM64.

Resultado

Los contenedores pudieron iniciarse correctamente.

Problema 4. Conflictos de puertos

Descripción

Varios servicios intentaban utilizar los mismos puertos dentro de Docker.

Causa

Diferentes contenedores compartían puertos HTTP ya utilizados.

Solución aplicada

Se modificaron manualmente los puertos publicados.

Ejemplo aplicado

8181

Resultado

Todos los servicios pudieron ejecutarse simultáneamente sin conflictos.

Problema 5. Error de conexión con base de datos

Descripción

Un contenedor no conseguía conectarse correctamente al servicio de base de datos.

Error detectado

php_network_getaddresses

Causa

El nombre del contenedor de base de datos estaba incorrectamente configurado.

Solución aplicada

Se corrigió el nombre del contenedor dentro de la configuración Docker.

Resultado

La conexión entre servicios quedó funcionando correctamente.

Problema 6. Contenedores detenidos automáticamente

Descripción

Algunos contenedores aparecían con estado:

exited - code 0

Causa

El servicio ejecutado dentro del contenedor finalizaba inmediatamente después de arrancar.

Solución aplicada

Se revisaron:

- Comandos de inicio
- Variables de entorno
- Configuración del contenedor

Resultado

Los servicios críticos quedaron funcionando correctamente en estado running.

Problema 7. Acceso web no disponible

Descripción

Algunos servicios no cargaban desde navegador web.

Causa

- Puerto incorrecto
- Contenedor detenido
- Problemas DNS
- Servicio no expuesto

Solución aplicada

Se verificó:

- Estado del contenedor
- Publicación de puertos
- Configuración DNS
- Redes Docker

Resultado

Los servicios quedaron accesibles desde navegador correctamente.

Aprendizajes obtenidos

La resolución de incidencias permitió mejorar conocimientos sobre:

- Docker
- Redes
- DNS
- Arquitectura ARM64
- Gestión de contenedores
- Monitorización
- Administración Linux

Resultado final

Todos los problemas encontrados durante el proyecto fueron solucionados correctamente, permitiendo desplegar una infraestructura estable y completamente funcional sobre Raspberry Pi utilizando Docker.

14. Mejoras Futuras

La infraestructura desarrollada permite seguir ampliando funcionalidades y añadir nuevos servicios orientados a entornos profesionales.

Gracias al uso de Docker y contenedores, el sistema puede escalar fácilmente incorporando nuevas herramientas de administración, seguridad y monitorización.

Posibles ampliaciones de la infraestructura

Seguridad y acceso

- Reverse Proxy con Nginx
- HTTPS con certificados SSL
- VPN interna para acceso remoto seguro
- Firewall avanzado para protección de servicios

Estas mejoras permitirían aumentar la seguridad y proteger el acceso a las aplicaciones desplegadas.

Administración y automatización

- Automatización de despliegues con Ansible
- Backups automáticos
- Gestión centralizada de usuarios
- Alta disponibilidad de servicios

La automatización reduciría tareas manuales y facilitaría el mantenimiento de la infraestructura.

Monitorización avanzada

- Alertas automáticas mediante correo electrónico
- Monitorización remota desde Internet
- Dashboards avanzados en Grafana
- Supervisión completa de recursos y contenedores

Estas herramientas permitirían detectar fallos rápidamente y mejorar el control del sistema.

Integración futura de GLPI

Como ampliación futura, se plantea integrar GLPI como plataforma de gestión IT dentro de la infraestructura.

GLPI permitiría centralizar la administración de equipos, incidencias y documentación técnica desde una única interfaz web.

Funciones principales de GLPI

- Gestión de incidencias
- Sistema de tickets
- Inventariado automático de equipos
- Administración de usuarios y dispositivos
- Centralización de documentación técnica

Beneficios de futuras ampliaciones

La incorporación de estas tecnologías permitiría transformar la infraestructura actual en un entorno mucho más cercano a una red empresarial real.

Mejoras obtenidas

- Mayor seguridad
- Mejor organización administrativa
- Automatización de tareas
- Gestión IT centralizada
- Monitorización avanzada
- Infraestructura más escalable y profesional

15. Anexos

Puertos utilizados

Servicio	Puerto
Portainer	9443
Pi hole	8099
Grafana	3000
Prometheus	9090
cAdvisor	8089
Node Exporter	9100
Netdata	19999
Nextcloud	8081

Direcciones IP utilizadas

Dispositivo	Dirección IP
Raspberry Pi	192.168.103.25
Clientes Windows	IP manual
DNS Principal	192.168.103.25

Configuración aplicada

Parámetro	Valor
IP	192.168.103.25
Máscara	255.255.255.0
Gateway	192.168.103.1
DNS	192.168.103.25

Comandos Docker importantes

Ver contenedores

docker ps

Ver logs

docker logs nombre_contenedor

Reiniciar contenedor

docker restart nombre_contenedor

Detener contenedores

docker compose down

Levantar contenedores

docker compose up -d

Servicios desplegados

- Portainer
- Pi hole
- Grafana
- Prometheus
- Netdata
- cAdvisor
- Node Exporter
- Nextcloud

Tecnologías utilizadas

- Raspberry Pi OS
- Docker
- Docker Compose
- Portainer
- Linux
- DNS local
- Monitorización

Conclusión

El desarrollo de esta infraestructura permitió implementar un entorno funcional de servicios y monitorización utilizando una Raspberry Pi 4 como servidor principal y Docker como plataforma de contenedores.

A lo largo del proyecto se configuró correctamente Raspberry Pi OS, se desplegaron múltiples servicios mediante Docker y se administró toda la infraestructura desde Portainer de forma centralizada y eficiente.

La integración de herramientas como Pi hole, Nextcloud, Prometheus, Grafana, Netdata, cAdvisor y Node Exporter permitió crear un entorno completo orientado a administración de sistemas, almacenamiento privado y monitorización avanzada.

Uno de los aspectos más importantes del proyecto fue demostrar que una Raspberry Pi puede utilizarse como servidor capaz de ejecutar múltiples servicios simultáneamente manteniendo un funcionamiento estable y eficiente dentro de una red local.

Además, el uso de contenedores Docker facilitó enormemente la instalación, organización y mantenimiento de cada servicio, aportando modularidad, aislamiento y escalabilidad a toda la infraestructura.

Durante el desarrollo también se trabajó con conceptos fundamentales de administración de sistemas y redes como:

- Gestión de servicios
- Configuración de red
- Virtualización ligera
- Monitorización
- Almacenamiento web
- Administración remota
- Gestión de contenedores

El proyecto permitió simular un entorno profesional real, aplicando conocimientos prácticos de infraestructura, redes y administración Linux en una solución compacta, económica y escalable.

Como resultado final, se obtuvo una plataforma completamente funcional capaz de ofrecer servicios web, monitorización y administración centralizada dentro de una arquitectura moderna basada en Docker.

